TradeAgent — Build Spec for an AI-Agent XAUUSD Scalping Bot on MT5

Oct 26, 2026 · @Nikhil

0. How to use this spec (read first)

This document is the single source of truth for building TradeAgent. It is written for an AI coding agent (Claude Opus in Claude Code) to build the whole system, step by step, with tests proving each step before the next one starts.

Instructions for the building agent

1. Read the whole spec before writing code. Where two sections conflict, the risk rules in section 7 win.
2. Build in the order of section 14 (Build plan). Do not start a step until the previous step's acceptance tests pass.
3. After each step: run all tests, commit to Git with the step number, and write a short `BUILD_LOG.md` entry (what was built, what passed, any open issue).
4. Stop and ask the human owner at every **HUMAN CHECKPOINT** marked in section 14. Never skip one.
5. Never invent trading rules. Rules come only from `config/rulebook.yaml` versions marked `approved: true` by the human trader.
6. Never let an LLM compute a price, stop, target or lot size. Agents call deterministic tools for every number (section 2.4).
7. Every order must pass through the risk engine (section 7). No code path may call MT5 `order_send` except `core/execution`, and only with a risk approval token.
8. Default to demo. Live trading requires `mode: live` in config, set by a human, plus the go-live checkpoint.
9. If something in this spec is unclear or impossible, write it as an open question in `BUILD_LOG.md` and pick the safer option.

Words used in this spec

Term	Meaning
Agent	An LLM with a role prompt and a fixed set of tools
Tool	A deterministic Python function an agent may call

Term	Meaning
Build-time agent	Agent that helps develop, test and review the system
Runtime agent	Agent that runs while the bot trades
Rulebook	`config/rulebook.yaml`: the approved setups the bot is allowed to trade
Human owner	Nikhil: final say on risk limits, go-live and compliance
Human trader	Approves every rulebook version before it can be used

1. Overview

TradeAgent is a team of AI agents that builds, tests and runs a fully automated XAUUSD scalping bot on MT5, where runtime agents analyse the market and place and manage trades directly on the MT5 account. Humans keep only three jobs: approve the trading rules, set the risk limits, and approve going live.

Key decisions

Decision	Choice	Why
Instrument	XAUUSD only	One market, done well
Style	Scalping, M1/M5 entries	Bias from H4/H1, zones from M15
Who builds it	Build-time AI agents, orchestrated by Claude Opus in Claude Code	Replaces the human dev, quant and QA roles
Who trades	Runtime AI agents place and manage trades on MT5	Owner's requirement
Where rules come from	Trader agent drafts, human trader approves	Agents can't be trusted to invent an edge alone
How numbers are made	Deterministic Python tools only	LLMs are unreliable at arithmetic; prices and sizes must be exact
Final gate on every order	Deterministic risk engine	Agents can request trades; they cannot bypass risk limits

Decision	Choice	Why
Platform	MT5 on our own Windows PC	The MetaTrader5 Python package needs Windows
Language	Python 3.11	Reuses the gold-scanner detectors
LLM provider	Pluggable: Gemini (free tier), local model via Ollama, or Claude API	Keeps cost near zero; swap without code changes

In scope

- Build-time agent team that writes, tests and reviews the code (section 2)
- Runtime agent team that analyses, decides, executes and manages trades (section 2)
- SMC concepts (BOS, CHoCH, OB, FVG, liquidity sweeps), supply/demand, S/R, candlestick and chart patterns
- Multi-timeframe analysis D1 / H4 / H1 / M15 / M5 / M1
- Session and news filters
- Position sizing, SL/TP, break-even, partials, trailing
- Hard risk limits and a kill switch
- Journal of every agent decision, dashboard, Telegram alerts and controls
- Backtesting and replay with the same code used live

Out of scope (v1)

- Other instruments
- Agents changing their own rules or risk limits
- Managing other people's money
- Mobile app

What "good" means

Success is proven on demo first: positive expectancy after spread and slippage, drawdown within limits, and every trade traceable to an approved rule and a logged agent decision.

2. Agent team

The human team is replaced by two agent teams: seven build-time agents that build and verify the system, and seven runtime agents that trade it, all working through a fixed set of deterministic tools.

2.1 What humans still do

Human	Job	Time needed
Human owner (Nikhil)	Approves risk limits, go-live, each capital step-up; does the legal check; keeps the Windows PC running	~1 hour/week after build
Human trader	Reviews and approves each rulebook version drafted by the Trader agent; spot-checks weekly trade reviews	~1–2 hours/week

No agent may change `rulebook.yaml` (approved versions), `config/risk.yaml`, or `mode: live`. Only humans edit these.

2.2 Build-time agents

These run inside Claude Code during development. The Orchestrator is the main Opus session; the others are sub-agents defined as files in `.claude/agents/` (one Markdown file per agent with its role prompt and allowed tools).

Agent	Replaces	Job	Output	Done when
Orchestrator	Team lead	Follows section 14 step by step, assigns work, enforces checkpoints	`BUILD_LOG.md`, commits	All steps pass their tests
Trader agent	Trader (drafting)	Interviews the human trader in plain language, turns answers into rulebook entries, labels example charts	`rulebook.draft.yaml`, labelled examples	Human trader marks the version `approved: true`

Agent	Replaces	Job	Output	Done when
Quant agent	Quant researcher	Builds backtester, runs backtests and walk-forward tests, checks for overfitting, writes reports	`reports/*.md`, metrics JSON	Gate 3 metrics computed and reported honestly
Developer agent	Full-stack dev	Writes engine, tools, agents, API, dashboard	Code + unit tests	Tests pass, reviewed
QA agent	QA engineer	Writes test cases, edge cases (requotes, disconnects, bad config), runs the full suite	`tests/`, QA report	No failing tests; edge-case list covered
Risk reviewer agent	Risk owner (review only)	Reviews every change to `risk/` and `execution/`; checks no path bypasses the risk engine	Review notes; block/approve	Zero bypass paths found
DevOps agent	Automation dev	Windows service setup, auto-restart, backups, CI, Telegram wiring	Scripts, setup guide	Bot survives reboot and crash tests

2.3 Runtime agents

Runtime agents are split by speed. LLM agents think only when there is something to decide; deterministic services do everything that must be fast or exact.

Agent	Type	Runs when	Job	May call
Supervisor agent	Deterministic + LLM summary	Always	Starts/stops others, health checks, kill switch, daily summary	All read tools, `pause_trading`, `send_alert`
Market Analyst agent	LLM	Each closed M5 candle, and on M1 when price is in an active zone	Reads detector output across timeframes; writes the market view (bias, active zones, what to watch)	Read tools only
Trade Decision agent	LLM	When a setup candidate passes the rulebook gates	Checks the candidate against the approved rulebook and market view; says TAKE or SKIP with reasons	Read tools, `compute_trade_plan`
Risk Guardian	Deterministic (no LLM)	Before every order	Applies section 7 limits; returns approval token or rejection	n/a (it is a tool)
Execution agent	Deterministic	On approval token	Sends order to MT5, verifies fill	`place_order`
Trade Manager agent	Deterministic rules + LLM for early-exit judgement	Each tick (rules) / each M1 close (LLM)	Break-even, partials, trailing; LLM may suggest early close on structure change	`modify_sl`, `close_partial`, `close_position`

Agent	Type	Runs when	Job	May call
Journal agent	LLM, after market close	Daily	Reviews the day's trades and skipped setups; writes a review for the human trader; proposes rule changes as drafts only	Read tools, `write_report`

This split means the agent does place and manage trades on MT5 on its own, but every number it uses comes from tools, and every order passes the Risk Guardian.

2.4 Tool catalogue

Agents act only through these tools. Each is a Python function with typed input/output (pydantic) and unit tests.

Tool	Returns	Used by
`get_candles(tf, n)`	OHLC + spread	Analyst, Decision
`get_structure(tf)`	Swings, BOS, CHoCH, trend state	Analyst, Decision
`get_zones(tf)`	OB, FVG, S/D, S/R zones with status	Analyst, Decision
`get_liquidity()`	Sweeps, equal highs/lows, session highs/lows, PDH/PDL	Analyst, Decision
`get_patterns(tf)`	Candlestick and chart patterns at zones	Analyst, Decision
`get_session_and_news()`	Current session, killzone, next high-impact event	All
`get_setup_candidates()`	Setups that passed rulebook gates, with score	Decision
`compute_trade_plan(candidate_id)`	`plan_id`, entry, SL, TP1, TP2, lots, R:R	Decision

Tool	Returns	Used by
`risk_check(plan_id)`	Approval token or rejection reason	Execution
`place_order(plan_id, token)`	Ticket, fill price, verified flag	Execution
`modify_sl(ticket, new_sl)`	OK/error (only risk-reducing moves allowed)	Trade Manager
`close_partial(ticket, pct)` / `close_position(ticket)`	Result	Trade Manager
`get_positions()` / `get_account()`	Open trades, equity, limits used	All
`log_decision(...)` / `send_alert(...)`	OK	All
`pause_trading(reason)`	OK	Supervisor

2.5 Agent output contracts

LLM agents must reply in strict JSON, validated by pydantic. Invalid JSON, a timeout (default 8 s), or an unknown `plan_id` = **SKIP**.

```
{
  "agent": "trade_decision",
  "candidate_id": "c_20261026_0814_01",
  "rule_id": "london_sweep_m1_choch_m15_ob",
  "rulebook_version": "1.2.0",
  "decision": "TAKE",
  "plan_id": "p_20261026_0814_01",
  "confidence": 0.72,
  "reasons": ["H1 bullish BOS", "fresh M15 OB", "Asia low swept", "M1
CHoCH"],
  "concerns": ["US data in 95 min"]
}
```

The agent never writes a price or lot size. It can only point to a `plan_id` that `compute_trade_plan` produced.

2.6 Prompt rules for every runtime agent

- Role, allowed tools, and output schema in the system prompt; nothing else.
- The approved rulebook is injected as data. The agent may only TAKE a setup that matches a rule ID in it.

- When unsure, SKIP. A skipped good trade costs nothing; a bad trade costs money.
- Prompts live in `agents/prompts/*.md`, versioned in Git; the prompt version is logged with each decision.
- If the LLM provider is down or slow, no new trades open. Open trades keep being managed by deterministic rules.

3. Cost and compliance

The bot can run at near-zero cost if runtime LLM calls use a free tier or a local model; the build itself uses Claude Opus in Claude Code, which needs a paid Claude plan.

Item	Cost	Notes
MT5 terminal, demo account, broker history	Free	From the broker
Python, libraries, database, FastAPI, React	Free	Open source
Telegram bot	Free	Alerts and controls
Windows PC	Free (already owned)	Must stay on 24/5
UPS + stable internet	Low, one-time	Strongly recommended
Build-time agents (Claude Opus in Claude Code)	Paid Claude plan	Only during development
Runtime LLM: Gemini free tier	Free within rate limits	Check current limits before relying on it
Runtime LLM: local model via Ollama	Free	Needs a capable GPU for acceptable speed
Runtime LLM: Claude API	Pay per use	Optional, best quality
Trading capital	Our choice	Start small after demo passes
Legal consult	One-time fee	See below

Keeping runtime LLM use small

- LLM agents are called only on events (a setup candidate, an M5 close, an open trade on M1 close), not on every tick.

- The bot must count calls per day and pause new entries before hitting a provider's limit.
- Same input = same cached answer, so replays and restarts don't repeat calls.

Hidden costs

- **Spread and commission** take a large share of each M1/M5 scalp. Every test includes them.
- **Slippage** around news. Assume worse fills than the backtest.
- **LLM latency.** A 2–8 s decision delay can move the entry on M1. Limit orders at the zone reduce this.

Compliance check (human owner, before going live)

Indian residents trading XAUUSD CFDs or margin forex through offshore brokers can face restrictions under RBI/FEMA rules, and RBI keeps an alert list of unauthorised forex platforms. This is not legal advice; a CA or legal advisor must confirm the broker and account setup before any real-money trading.

- [ ] Book a consult with a CA / legal advisor
- [ ] Confirm the broker is allowed for our situation
- [ ] Confirm tax treatment of profits

4. Tech stack

Everything runtime runs on one Windows PC: Python talks to MT5 directly, runtime agents call an LLM through one provider interface, and build-time agents run in Claude Code.

Layer	Tool	Purpose
Broker platform	MetaTrader 5 terminal	Prices, history, order execution
MT5 bridge	`MetaTrader5` Python package	Candles, ticks, orders
Core language	Python 3.11	All engine and agent code
Data	pandas, numpy	Candle frames, calculations
Indicators	pandas-ta	ATR, volatility helpers
Detectors	Our own code (port gold-scanner)	BOS, CHoCH, OB, FVG, sweeps, zones, patterns

Layer	Tool	Purpose
Config	YAML + pydantic	`rulebook.yaml`, `risk.yaml`, `settings.yaml`
Runtime agent loop	Plain Python + LLM tool-calling (no heavy framework)	Easy to test, fewer moving parts
LLM interface	`agents/llm/` provider adapter: Gemini, Ollama, Claude	Swap provider in config
Agent tools	Python functions + pydantic schemas	The only way agents act
Build-time agents	Claude Code with sub-agents in `.claude/agents/`	Build, test, review
Backtesting	Own event-replay engine (same code as live)	Honest tests
Database	SQLite (v1), PostgreSQL later	Candles, signals, agent decisions, trades
API	FastAPI	Dashboard and controls
Dashboard	React + lightweight-charts	Chart with zones, trades, agent reasoning
Alerts and control	python-telegram-bot	Alerts, /status, /pause, /kill
News calendar	Free economic calendar feed	News blackout
Service	NSSM (Windows service)	Auto-start, auto-restart
Testing	pytest, plus recorded LLM responses for repeatable tests	Every tool, rule and agent contract
Version control and CI	Git + GitHub Actions	Tests on each push

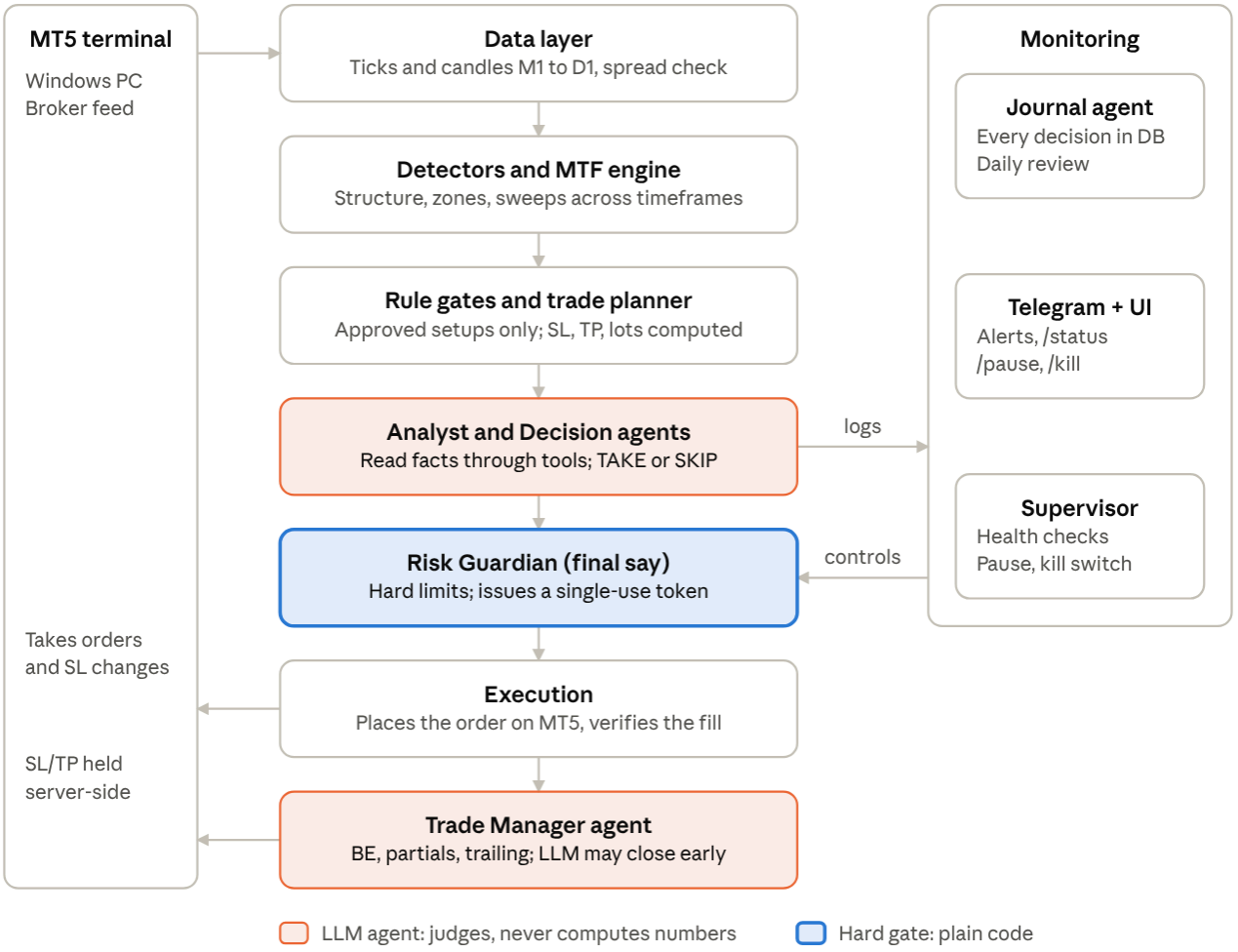
Notes

- Agents use the provider's native tool-calling. If a provider has no tool-calling, the adapter asks for JSON and maps it to tool calls.
- Tests never call a live LLM; they replay recorded responses so results are repeatable.
- If no news feed is available, the bot does not trade near known event times (fail safe).

5. System architecture

TradeAgent has three layers: a deterministic engine that reads the market and does all the maths, an agent layer that judges and decides, and a hard risk gate between the agents and MT5.

Agents decide, but every order passes a hard risk gate before MT5



TradeAgent architecture · engine, agents, risk gate

Detectors and the multi-timeframe engine turn candles into facts (zones, structure, setup candidates). The Market Analyst and Trade Decision agents read those facts through tools and decide TAKE or SKIP. A TAKE becomes an order only after the Risk Guardian approves it. The backtester swaps the data layer for a replay feed and the broker for a simulator; everything else is the live code.

Component	Layer	Input	Output
Data layer	Engine	MT5 ticks and candles	Clean candle frames per timeframe

Component	Layer	Input	Output
Detectors	Engine	Candles	Structure events, zones, patterns
Multi-timeframe engine + rulebook gates	Engine	Detector output, approved rulebook	Setup candidates with scores
Market Analyst agent	Agent	Tools: structure, zones, liquidity, news	Market view (JSON)
Trade Decision agent	Agent	Candidate + market view + rulebook	TAKE (with `plan_id`) or SKIP
Trade planner (`compute_trade_plan`)	Engine	Candidate	Entry, SL, TP, lots, R:R
Risk Guardian	Gate	Plan + account state + limits	Approval token or rejection
Execution	Engine	Plan + token	Verified MT5 position
Trade Manager agent	Engine + agent	Open positions, new candles	SL moves, partials, exits
Supervisor agent	Ops	Health of everything	Pause, kill, alerts
Journal / Telegram / Dashboard	Ops	Events from all parts	History, alerts, controls

6. Trading logic

The bot trades only setups in an approved rulebook version, and every setup must be defined so that code and two different readers would mark the same candles.

6.1 The rulebook

The Trader agent drafts the rulebook; the human trader approves it. The flow:

1. Trader agent asks the human trader to describe a setup in plain words (chat or voice notes transcribed to text).
2. Trader agent writes it in the format below and asks follow-up questions for anything vague (e.g. "is a sweep a wick beyond the low, or a close beyond it?").
3. Trader agent finds 30+ historical examples the rule would mark and shows them as chart images for the trader to confirm or reject.

4. Human trader sets `approved: true` and a version number. Only approved versions load at runtime.
5. Journal agent may propose changes later, but only as a new draft that goes back to step 4.

Each rulebook entry has these fields:

Field	Example
Setup name	"London sweep + M1 CHoCH into M15 OB"
Higher-timeframe bias rule	H1 last BOS bullish and price above H4 midpoint
Zone rule	Price inside an unmitigated M15 bullish order block or FVG
Trigger rule	M1 sweeps the last swing low, then closes above the CHoCH level
Allowed sessions	London open, NY open
Filters	Spread under limit, no high-impact news within the blackout window
Entry	Limit at OB 50% or market on trigger close
Stop loss	Below sweep low + buffer
Take profit	Next opposing liquidity / zone; TP1 and TP2
Minimum R:R	e.g. 1:2
Invalidation	Close beyond the zone before entry; setup older than N candles

Any rule that can't be written this precisely stays out of the rulebook and goes on a review list in `rulebook.draft.yaml`. Each entry also stores `id`, `version`, `approved`, `approved_by` and `approved_at`.

6.2 Detectors

Each detector reads candles and outputs tagged events or zones with timestamps, prices and a strength score. They are pure functions, unit-tested on hand-labelled charts.

Detector	Output	Timeframes
Swing points	Swing highs/lows with confirmation length	All

Detector	Output	Timeframes
Market structure	BOS, CHoCH, current trend state	D1, H4, H1, M15, M5, M1
Order blocks	OB zone, mitigated or not	H1, M15, M5
Fair value gaps	FVG zone, filled percent	M15, M5, M1
Liquidity	Equal highs/lows, session highs/lows, PDH/PDL, sweeps	All
Supply / demand	Base-and-departure zones, freshness	H4, H1, M15
Support / resistance	Levels from repeated reactions and round numbers	D1, H4, H1
Candlestick patterns	Engulfing, pin bar, inside bar, etc. at a zone	M5, M1
Chart patterns	Double top/bottom, flags, wedges (small v1 set)	M15, H1
Session / time	Asia, London, NY ranges; killzones	M1
Volatility	ATR, range expansion, spread state	M1, M5

The gold-scanner's five detectors (CHoCH, BOS, liquidity sweep, OB tap, FVG fill) are the starting point; they get ported into this structure and re-tested.

6.3 Multi-timeframe flow

1. **D1 / H4: context.** Trend direction, key S/R and supply/demand zones, premium vs discount.
2. **H1: bias.** Bullish, bearish or no-trade. No-trade means the bot sits out.
3. **M15: zone.** Find the unmitigated OB, FVG or S/D zone in the bias direction.
4. **M5: setup.** Price reaches the zone; look for a sweep or structure shift.
5. **M1: trigger.** CHoCH or confirming candle gives the exact entry.

6.4 Confluence score

Each setup gets a score from the confluences present. Weights are drafted by the Trader agent, approved by the human trader, then tuned only on in-sample data by the Quant agent.

Confluence	Example weight
HTF bias agrees	Required (gate)
Inside fresh M15 zone	Required (gate)
Liquidity sweep before trigger	+2
M1 CHoCH confirmed	Required (gate)
Zone overlaps H1/H4 S/R or S/D	+2
In London/NY killzone	+1
FVG inside OB	+1
Confirming candlestick	+1

A trade fires only when all gates pass and the score meets the threshold (e.g. 4+). The weights above are placeholders until the human trader approves them.

7. Risk management engine

The risk engine (Risk Guardian) has the final say: it is plain deterministic code, no agent can change its limits, and no order reaches MT5 without its approval token.

7.1 Position sizing

Lot size comes from a fixed percentage of account equity and the stop distance:

```
\text{lots} = \frac{\text{equity} \times \text{risk\%}}{\text{SL distance (price)} \times \text{contract size}}
```

- Contract size, tick value, min lot, lot step and max lot are read live from MT5 `symbol_info`, never hard-coded (they differ by broker).
- Lot is rounded **down** to the lot step. If it falls below min lot, the trade is skipped.
- Starting risk per trade: 0.25%–0.5% of equity (set by the human owner in risk.yaml).

7.2 Stop loss and take profit

- **SL:** placed beyond the structure that invalidates the setup (sweep low/high or zone edge) plus a buffer of max(spread × k, ATR(M1) × k).
- **SL limits:** skip the trade if SL is smaller than the minimum (noise) or larger than the maximum (too wide for a scalp). Both values in the rulebook.
- **TP1:** nearest opposing liquidity or zone. **TP2:** next HTF level.

- **R:R gate:** if TP1 gives less than the minimum R:R after spread, the trade is rejected.

7.3 Account limits

Limit	Starting value (to confirm)	Action when hit
Risk per trade	0.5%	Hard cap on size
Max open trades	5	New signals rejected
Max total open risk	2% (sum of all open trades' risk)	New signals rejected; stops 5 open trades × 0.5% exceeding the daily loss limit
Max trades per day	50	Stop for the day
Daily loss limit	2%	Close all, stop for the day
Weekly loss limit	5%	Stop for the week, review required
Consecutive losses	3	Pause 1 session
Max drawdown from peak	10%	Full stop, manual restart only
Max spread	Set from broker's normal gold spread	Skip entry

7.4 Kill switch

Trading stops automatically, and open trades are protected or closed, when:

- daily or max drawdown limit is hit
- price data is stale (no new tick for N seconds in market hours)
- MT5 connection is lost and doesn't recover within N seconds
- an order result doesn't match what was sent (wrong size, missing SL)
- someone sends /kill on Telegram or presses the dashboard button

Every order is sent with its SL attached server-side, so a crash of our PC never leaves a trade unprotected.

7.5 Agents and risk

- Limits live in `config/risk.yaml`, read-only to all agents and to the agent process (file permission).
- The approval token is single-use, tied to one `plan_id`, and expires after 30 s.

- `place_order` rejects any call without a valid token, any volume or SL that differs from the plan, and any order with no SL.
- `modify_sl` only accepts moves that reduce risk.
- QA agent must include a test that tries to bypass the Risk Guardian from every agent and tool; all attempts must fail.

8. Execution and trade management

When the Trade Decision agent says TAKE and the Risk Guardian approves, the Execution agent places the order on MT5 with SL and TP attached and verifies the fill; the Trade Manager agent then manages it by fixed rules, with the LLM allowed only to suggest closing early.

8.1 Placing an order

1. Re-check spread, symbol trading status and free margin just before sending.
2. Build the request: `order_type`, `volume`, `price`, `sl`, `tp`, `deviation` (max slippage), `magic` number (identifies our bot), `comment` (setup ID).
3. Use `order_check()` first, then `order_send()`.
4. Read the result code. On requote or price change, retry once with fresh price; otherwise log and drop.
5. Confirm the opened position matches: volume, SL present, TP present. If not, fix or close immediately and alert.

Entry types: market order on trigger close, or limit order at the zone with an expiry (cancel if not filled within N M1 candles or if the setup is invalidated).

8.2 Managing open trades

Event	Action
Price reaches 1R	Move SL to break-even + small buffer
TP1 hit	Close 50% (configurable)
New M1/M5 structure forms in our favour	Trail SL behind the latest protected swing
Opposite CHoCH on M5 against the trade	Close early
Trade open longer than max time	Close at market
High-impact news approaching	Close or tighten SL (rulebook decides)
Friday close / session end	Close all scalps

All SL changes only ever move in the direction that reduces risk.

The Trade Manager's LLM check runs on each M1 close for open trades. It may only return HOLD or CLOSE_EARLY with reasons; CLOSE_EARLY is executed through `close_position`. It can never widen a stop, add to a position, or reverse it.

8.3 Idempotency and safety

- Each signal has a unique ID; the engine never sends two orders for the same ID.
- On restart, the bot reads open positions by magic number and resumes managing them before looking for new trades.
- Only one process may trade at a time (a lock file or DB lock).

9. How the bot runs

The bot runs a fast tick loop that guards open trades with plain code, and an event loop on each closed candle where the engine finds candidates and the agents decide.

9.1 Startup

1. Load and validate config, `risk.yaml`, and the latest **approved** rulebook (none approved = refuse to start).
2. Connect to MT5; check account, symbol, trading permission, and `mode` (demo/live).
3. Check the LLM provider responds; if not, start in manage-only mode (no new entries).
4. Load history for all timeframes; rebuild zones and structure.
5. Load today's news calendar.

6. Resume managing open positions with our magic number.
7. Supervisor sends "bot started" to Telegram with mode, rulebook version, and risk summary.

9.2 Tick loop (every ~250 ms)

1. Read latest tick and spread.
2. Check health: data fresh, MT5 connected, kill switch not active.
3. For each open trade, run trade management rules (section 8.2).

9.3 Candle loop (on each closed M1 candle)

1. Append the new candle; update higher timeframes when they close.
2. Re-run detectors on changed timeframes; update bias and active zones.
3. On each M5 close: Market Analyst agent refreshes the market view.
4. Engine checks approved rulebook gates and returns setup candidates with scores.
5. For each candidate: `compute_trade_plan` makes the plan; Trade Decision agent returns TAKE or SKIP (8 s timeout = SKIP).
6. On TAKE: Risk Guardian checks the plan; on approval, Execution agent places and verifies the order.
7. For open trades: Trade Manager LLM check (HOLD / CLOSE_EARLY).
8. Log every candidate, agent input/output, decision, and result; send Telegram alerts.

9.4 Daily cycle

- **Before Asia:** refresh news calendar, reset daily counters, post the day's key levels to Telegram.
- **End of NY session:** close remaining scalps, post daily summary (trades, win rate, R gained, drawdown); Journal agent writes the daily review for the human trader.
- **Weekend:** bot idle; the team reviews the week's journal.

9.5 A trade, start to finish (example)

At London open the engine sees H1 bullish, price in a fresh M15 bullish OB, the Asia low swept, and an M1 CHoCH: it matches approved rule `london_sweep_m1_choch_m15_ob` with score 4. `compute_trade_plan` returns SL under the sweep low plus buffer, TP1 at the Asia high, R:R 1:2.4, 0.5% risk. The Trade Decision agent reads the Analyst's view, notes US data in 95 minutes (outside the blackout), and returns TAKE with the `plan_id`. The Risk Guardian checks open risk and daily limits and issues a token. The Execution agent places

the order and confirms SL and TP are on the position. At 1R the stop moves to break-even; TP1 closes half; the rest trails until stopped. The journal stores every step, including the agents' reasons.

10. Data model

Nine tables hold everything needed to replay, audit and review any decision, human or agent.

Table	Key columns	Purpose
`candles`	timeframe, time, open, high, low, close, tick_volume, spread	Local market data
`zones`	id, type (OB/FVG/SD/SR), timeframe, top, bottom, created_at, mitigated_at, strength	Zones found
`structure_events`	id, type (BOS/CHoCH/sweep), timeframe, price, time, direction	Structure history
`candidates`	id, rule_id, rulebook_version, time, direction, score, confluences (JSON), plan_id	Setups that passed rulebook gates
`agent_runs`	id, agent, prompt_version, provider, model, input (JSON), output (JSON), tool_calls (JSON), latency_ms, valid, error	Every LLM agent call
`decisions`	candidate_id, agent_run_id, decision (TAKE/SKIP), reasons, risk_result, reject_reason	Final outcome per candidate
`trades`	id, candidate_id, ticket, entry, sl, tp1, tp2, lots, risk_pct, opened_at, closed_at, exit_price, pnl, r_multiple, exit_reason	Every demo or live trade
`trade_events`	trade_id, time, event (BE/partial/trail/close/early_close), old_value, new_value, agent_run_id	Changes to open trades
`system_events`	time, level, component, message	Starts, errors, disconnects, kill switch

Rules:

- `rulebook_version`, `prompt_version` and `config_hash` are stored with every decision and trade.

- SKIP decisions are kept; they show whether agents are skipping good trades.
- Database backed up daily to free cloud storage.

11. Backtesting and validation

No strategy goes live until it passes four gates in order: detector accuracy, honest backtest, out-of-sample test, and 8+ weeks of demo trading; the Quant agent runs the tests and the human owner signs off.

11.1 Gate 1: detector accuracy

The Trader agent prepares at least 200 examples per detector; the human trader confirms or corrects the labels. Each detector must match the confirmed labels (target agreed, e.g. 90%+) before strategy testing begins.

11.2 Gate 2: honest backtest

- Same engine code as live; only the data source changes (replay instead of MT5).
- Candles are fed one at a time; the engine never sees future data.
- Real spread per candle, plus commission, plus slippage assumption.
- M1 entries need M1 or tick data; for intra-candle SL/TP order, assume the worst case.

11.3 Gate 3: out-of-sample and walk-forward

- Split history: tune on the oldest ~70%, test once on the newest ~30%.
- Walk-forward: re-tune on rolling windows and test on the next window.
- Keep the number of tuned parameters small. More knobs means more overfitting.

11.3a Testing with LLM agents in the loop

LLM answers vary and cost calls, so backtests run in two modes:

- **Rules-only mode:** every candidate that passes the rulebook gates is taken. This measures the raw edge of the rules and runs fast over years of data.
- **Agent mode:** the agents decide on a sample (e.g. the last 3–6 months). Responses are cached by input hash so reruns are repeatable and free.

The agents must improve on rules-only mode (fewer losing trades, same or better expectancy). If they don't, run with agents in SKIP-only advisory mode or switch them off.

11.4 Gate 4: demo forward test

- Minimum 8 weeks, ideally 12, on a demo account with the live setup.

- Journal agent reviews every trade taken and every SKIP each week; the human trader spot-checks its review.
- Compare demo results with backtest over the same period; a big gap means a bug or bad assumption.

11.5 Metrics we track

Metric	Why it matters
Number of trades	Too few = results not meaningful (aim for 100+)
Win rate	With average R, gives expectancy
Average R per trade (expectancy)	Must be positive after costs
Profit factor	Gross profit ÷ gross loss; above 1.3 preferred
Max drawdown	Must stay inside the risk limit
Longest losing streak	Sets realistic expectations and pause rules
Results by session and setup	Shows which setups actually earn
Backtest vs demo gap	Detects bugs and fill problems

11.6 Going live

Start with the smallest lot size the broker allows. Increase risk only after each block of ~50 live trades stays within expectations, and only with human owner sign-off.

12. Monitoring and operations

The Supervisor agent watches every part of the bot, alerts the human owner through Telegram, and fails safe whenever something breaks.

12.1 Telegram

Message / command	When
Trade opened / closed	Every trade, with setup, entry, SL, TP, lots, R
Trade managed	BE moved, partial taken, trailed
Warning	Spread spike, data delay, reconnect
Critical	Kill switch fired, order mismatch, crash

Message / command	When
Daily summary	End of NY session
`/status`	Bot state, open trades, today's P/L and limits used
`/pause`, `/resume`	Stop/allow new entries (open trades still managed)
`/kill`	Close everything and stop

Commands accept only whitelisted Telegram user IDs.

12.2 Dashboard

- Live XAUUSD chart with current zones, structure marks and open trades
- Today's signals: taken and rejected, with reasons
- Equity curve, drawdown, stats by setup and session
- Controls: pause, resume, kill

12.3 Failure handling

Failure	Bot behaviour
MT5 disconnects	Retry for N seconds; no new trades; alert. SLs are on broker side.
Python crash	Windows service restarts it; on start it resumes open trades
Windows update / reboot	Updates scheduled for weekends only; auto-login and auto-start
Power cut	UPS covers short cuts; trades protected by broker-side SL
Internet down	Same as disconnect; consider mobile hotspot failover
News data unavailable	No trading within default windows around known event times
Bad config pushed	Validation refuses to start; previous version stays running

Agent-specific failures:

Failure	Bot behaviour
LLM provider down or rate-limited	No new entries; open trades managed by rules; alert
LLM reply invalid or too slow	That candidate is SKIPPED and logged
Agent asks for a tool it isn't allowed	Call refused, logged, counted; 3 in a day = pause and alert
Agent TAKE rate far above backtest	Supervisor pauses new entries and alerts (possible prompt or model drift)

12.4 Operating routine

- **Daily:** Supervisor summary + Journal agent review on Telegram; human owner glances at both.
- **Weekly:** human trader spot-checks the Journal agent's review and any proposed rule drafts.
- **Rule or prompt changes:** go through Git, are re-backtested by the Quant agent, reviewed by the Risk reviewer agent, and approved by the human trader (rules) or human owner (prompts, code) before deployment.

13. Project structure and coding standards

One Git repo, split by layer: engine code knows nothing about agents, agents act only through `tools/`, and the backtester reuses the live engine.

```
tradeagent/
├── .claude/
|   └── agents/                # build-time sub-agents: trader, quant,
developer, qa, risk-reviewer, devops
├── config/
|   ├── settings.yaml          # mode (demo/live), MT5, Telegram, LLM
provider
|   ├── risk.yaml              # limits — humans only
|   ├── rulebook.yaml          # approved versions only
|   └── rulebook.draft.yaml    # Trader/Journal agent drafts
├── core/
|   ├── data/                  # MT5 feed, replay feed (same interface)
|   ├── detectors/             # structure, ob, fvg, liquidity, sd, sr,
candles, patterns
|   ├── mtf/                   # bias, zones, triggers
|   ├── rules/                 # rulebook loader, gates, scoring
```

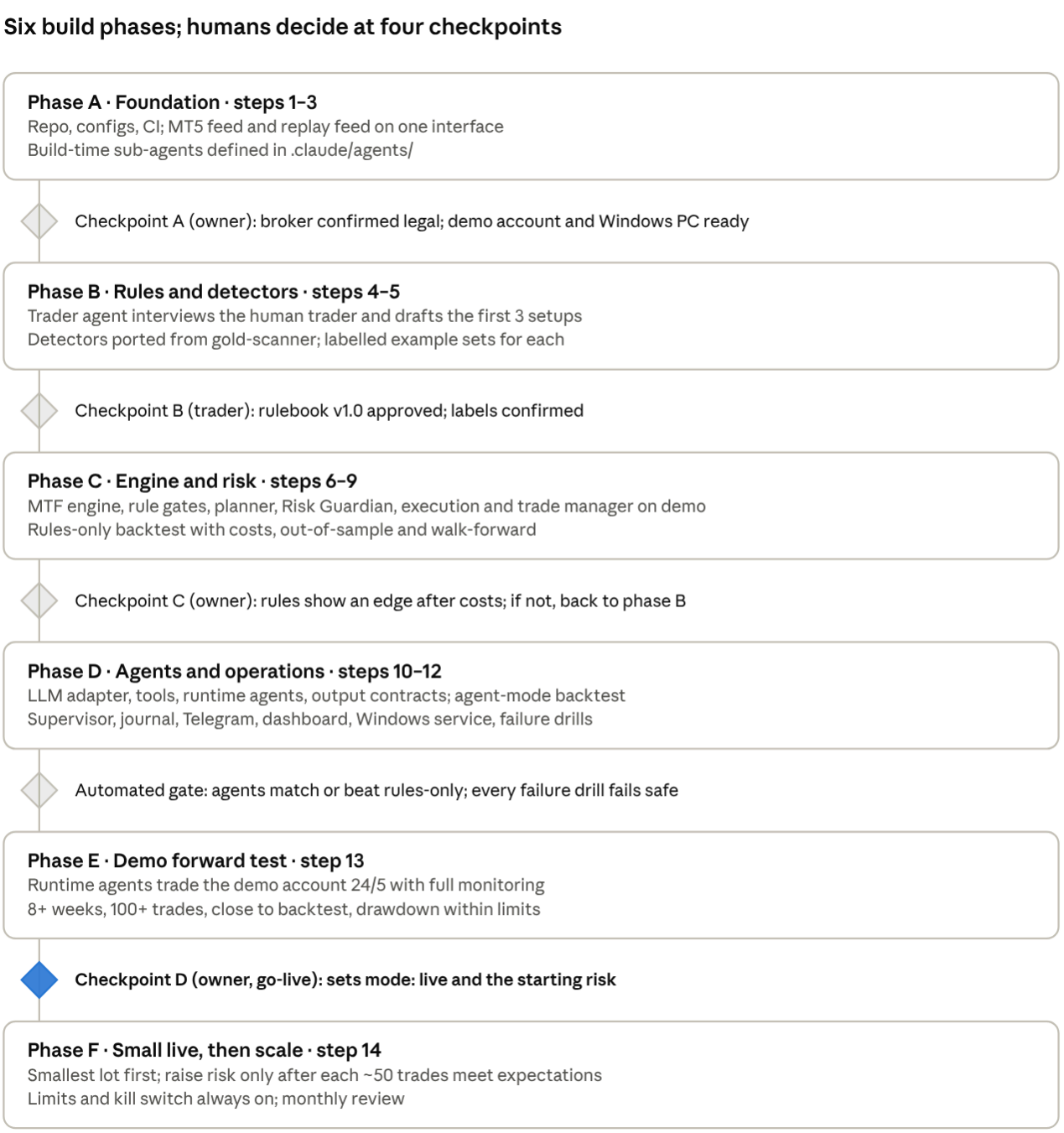
```
|   ├── planner/               # compute_trade_plan: SL, TP, lots
|   ├── risk/                  # Risk Guardian, tokens, kill switch
|   ├── execution/             # the ONLY module that calls order_send
|   └── manager/               # BE, partials, trailing
├── tools/                     # agent-callable tools (wrap core/, pydantic
I/O)
├── agents/
|   ├── llm/                   # provider adapters: gemini, ollama, claude;
cache; call counter
|   ├── prompts/               # analyst.md, decision.md, manager.md,
journal.md
|   ├── schemas/               # output contracts
|   └── runtime/               # supervisor, analyst, decision, manager,
journal
├── services/
|   ├── journal/  alerts/  news/  api/
├── dashboard/                 # React app
├── backtest/                  # replay runner, rules-only and agent modes,
reports
├── tests/                     # unit, labelled-chart, recorded-LLM, bypass
tests
├── BUILD_LOG.md
└── run_bot.py
```

Standards

- **One feed interface** for live and replay; this is what keeps backtests honest.
- **Pure detectors:** candles in, events out; no MT5 calls.
- **No magic numbers:** every threshold is in YAML.
- **Prices rounded to symbol digits** before any order.
- **Typed everything:** pydantic models for candidates, plans, decisions, orders, trades.
- **Tests required** for every detector, tool, risk rule and agent contract; CI runs on each push.
- **Review:** any change to `risk/`, `execution/` or `tools/` needs the Risk reviewer agent's approval and the human owner's OK.

14. Build plan

The building agent works through 14 steps in six phases; each step ends with passing acceptance tests, and four human checkpoints decide whether the build continues.



Build plan · 6 phases, 4 human checkpoints

14.1 Steps and acceptance tests

Step	Build	Lead agent	Acceptance tests (all must pass)
1	Repo, folder structure, config schemas, CI, `BUILD_LOG.md`	Developer	CI green; invalid config refuses to load

Step	Build	Lead agent	Acceptance tests (all must pass)
2	Data layer: MT5 feed + replay feed (same interface)	Developer	Pulls XAUUSD M1–D1 on demo; replay and live give identical candles for the same period
3	Build-time sub-agent files in `.claude/agents/`	Orchestrator	One file per agent with role, allowed tools, done criteria
	HUMAN CHECKPOINT A	Human owner	Broker confirmed legal; demo account and Windows PC ready
4	Rulebook interview; draft first 3 setups	Trader	`rulebook.draft.yaml` passes schema; 30+ chart examples per setup
5	Detectors (port gold-scanner first) + labelled sets	Developer, Trader	Gate 1 accuracy on labelled sets
	HUMAN CHECKPOINT B	Human trader	Rulebook v1.0 approved; labels confirmed
6	Multi-timeframe engine, rule gates, scoring	Developer	Candidates match rulebook examples; no look-ahead test
7	Planner + Risk Guardian	Developer, Risk reviewer	Sizing correct for several symbol specs; every limit in 7.3 tested; bypass tests all fail
8	Execution + deterministic trade manager on demo	Developer, QA	Open/modify/partial/close on demo; restart resumes trades; missing-SL fix; requote and disconnect tests
9	Backtester, rules-only mode + reports	Quant	Gate 2 and Gate 3 report with costs
	HUMAN CHECKPOINT C	Human owner	Rules show positive expectancy after costs out-of-sample; if not, back to step 4
10	LLM adapter, tools, runtime agents, prompts, output contracts	Developer, QA	Recorded-response tests; invalid JSON, timeout, unknown `plan_id` all give SKIP; tool permissions enforced
11	Agent-mode backtest	Quant	Agents match or beat rules-only expectancy on the sample period

Step	Build	Lead agent	Acceptance tests (all must pass)
12	Supervisor, journal, Telegram, dashboard, Windows service	DevOps, Developer	Crash, reboot, MT5 disconnect, LLM outage drills all fail safe
13	Demo forward test, 8+ weeks	Quant, Journal	Gate 4: 100+ trades, close to backtest, drawdown within limits
	HUMAN CHECKPOINT D (go-live)	Human owner	Sets `mode: live` and starting risk
14	Small live, then scale	Supervisor, Quant	Each ~50 trades within expectations before any risk increase

14.2 How to give this to the building agent

Open Claude Code in an empty repo on the Windows PC, add this spec as `SPEC.md`, and give one instruction: "Build TradeAgent by following SPEC.md section 0 and section 14, one step at a time; stop at every human checkpoint." Expect the build to take many sessions; the `BUILD_LOG.md` and Git history let any new session continue where the last one stopped.

15. Risks, open questions, next steps

The biggest risks are rules with no real edge, agents behaving differently live than in tests, and scalping costs eating the profit.

15.1 Risks

Risk	Impact	Mitigation
Rules have no real edge	Losing bot, however good the agents	Checkpoint C stops the build if rules-only backtest fails
Trader agent writes vague or wrong rules	Bot trades differently from the trader	Human trader approves every version; chart examples
LLM gives different answers on the same input	Unrepeatable results	Low temperature, strict JSON, caching, recorded-response tests

Risk	Impact	Mitigation
Agents add no value or make it worse	Lower profit	Agent-mode vs rules-only comparison; agents can be switched off
Prompt or model change shifts behaviour	Silent drift	Prompt version logged; TAKE-rate alarm; re-backtest on any change
LLM outage or rate limit	Missed trades	Manage-only mode; call counter; provider fallback
Overfitting	Good backtest, losing live	Few parameters, out-of-sample, walk-forward, 8+ weeks demo
Spread, slippage, LLM latency on M1	Edge disappears	Costs in every test; spread filter; limit entries
Execution bug	Large loss	Risk token, post-fill check, bypass tests, server-side SL
Single Windows PC fails	Stuck trades	Broker-side SL, UPS, auto-restart, alerts
One-shot build hides bugs	Broken system that looks finished	Step-by-step build with acceptance tests and checkpoints
Regulatory issue	Legal or account problems	Legal consult at checkpoint A

15.2 Open questions

- Which broker, and is it allowed for us?
- Starting capital and risk per trade?
- Which runtime LLM provider: Gemini free tier, local Ollama (GPU available?), or Claude API?
- Which 3 setups go into rulebook v1?
- How much M1/tick history can we get from the broker (2+ years needed)?

15.3 Next steps

- [ ] Human owner: book the legal consult and pick a broker
- [ ] Human owner: set up the Windows PC, MT5 demo account and Claude Code
- [ ] Human trader: block 2 hours for the first Trader agent interview

- [ ] Put this spec in the repo as `SPEC.md` and start step 1